

Урок 10. JS — `reduce()` и тег `template`

Метод `reduce()`

Метод `reduce()` позволяет пройти по массиву и «свернуть» его в одно итоговое значение — например, посчитать сумму всех элементов. Синтаксис:

```
const result = array.reduce((accumulator, currentValue, index, array) => {  
  // логика вычислений  
  return accumulator;  
}, initialValue);
```

В колбэк передаются четыре аргумента:

- `accumulator` (аккумулятор) — переменная, в которой накапливается результат. Ей присваивается то, что функция вернула на предыдущем шаге.
- `currentValue` — текущий обрабатываемый элемент массива.
- `index` (необязательно) — индекс текущего элемента.
- `array` (необязательно) — сам массив, который перебирается.

Как это работает изнутри

Аналогия из жизни: представьте, что вы считаете мелочь в копилке. Берёте первую монету (`initialValue` или первый элемент), а затем поочерёдно прибавляете к текущей сумме (`accumulator`) достоинство следующей монеты (`currentValue`). В итоге на руках остаётся одно число — общая сумма денег.

Важность `initialValue`

- Если `initialValue` передан: на первой итерации `accumulator` равен этому значению, а `currentValue` — первому элементу массива. Цикл выполнит столько шагов, сколько элементов в массиве.
- Если `initialValue` НЕ передан: `accumulator` автоматически станет равен первому элементу массива, а `currentValue` — второму. Первая итерация (для индекса 0) будет пропущена.

⚠ **Важно:** всегда лучше явно указывать `initialValue`. Если массив окажется пустым, а начальное значение не задано, JavaScript выбросит ошибку `TypeError`.

Пример: сложение элементов

```
const numbers = [1, 2, 3, 4, 5];  
  
const sum = numbers.reduce((acc, num) => {  
  return acc + num;  
}, 0);  
  
console.log(sum); // 15
```

- `acc` — аккумулятор метода, хранит значение предыдущей итерации.
- `num` — текущий элемент массива.

- `0` — начальное значение (может быть также `{}` или `[]`, смотря что нужно накопить).

Пример: поиск наибольшего значения

```
const numbers = [1, 2, 3, 4, 5];

const max = numbers.reduce((acc, num) => {
  return num > acc ? num : acc;
}, 0);

console.log(max); // 5
```

Пример: собрать значения одного ключа из массива объектов

```
const users = [
  { email: "test1@gmail.ru", name: "test1" },
  { email: "test2@gmail.ru", name: "test2" },
];

const names = users.reduce((acc, user) => {
  acc.push(user.name);
  return acc;
}, []);

console.log(names); // ['test1', 'test2']
```

Ter <template>

Ter `template` хранит разметку HTML, которая не отображается на странице, пока не будет вставлена через JS. Это удобно при работе с данными с backend, которые нужно отобразить на странице и у которых повторяющаяся структура в большом количестве — например, карточки множества пользователей. У них одинаковые поля, но разные значения, и таких пользователей может быть много. Вместо того чтобы прописывать разметку каждого пользователя вручную в HTML, можно один раз описать шаблон и вставлять пользователей через JS — это заметно сокращает объём кода.

Разметка в HTML

```
<body>
  <ul id="student-list"></ul>

  <template id="student-template">
    <li class="student-container">
      <p class="student-name"></p>
      <p class="student-surname"></p>
      <p class="student-age"></p>
      <p class="student-country"></p>
      <p class="student-city"></p>
      <p class="student-height"></p>
    </li>
  </template>
</body>
```

Массив данных в JS

```
const students = [
  {
    name: "Амир",
    surname: "Арифов",
    age: 23,
    country: "Россия",
    city: "Каспийск",
    height: 168,
  },
  {
    name: "Иван",
    surname: "Петров",
    age: 20,
    country: "Россия",
    city: "Новосибирск",
    height: 185,
  },
  {
    name: "Данил",
    surname: "Саидов",
    age: 33,
    country: "Дагестан",
    city: "Хасавюрт",
    height: 175,
  },
  {
    name: "Шамиль",
    surname: "Мусин",
    age: 35,
    country: "Казахстан",
    city: "Алматы",
    height: 175,
  },
];
```

Вставка данных через шаблон

```
const studentTemplate = document.getElementById("student-template");
const studentList = document.getElementById("student-list");

students.forEach((student) => {
  const studentClone = studentTemplate.content.cloneNode(true);

  studentClone.querySelector(".student-name").textContent =
    `Имя: ${student.name}`;
  studentClone.querySelector(".student-surname").textContent =
    `Фамилия: ${student.surname}`;
  studentClone.querySelector(".student-age").textContent =
    `Возраст: ${student.age}`;
  studentClone.querySelector(".student-country").textContent = student.country;
  studentClone.querySelector(".student-city").textContent = student.city;
  studentClone.querySelector(".student-height").textContent = student.height;

  studentList.appendChild(studentClone);
});
```

Обратите внимание: для подстановки `${student.name}` внутри строки нужны именно обратные кавычки ``...`` (шаблонная строка), а не обычные `"..."` — иначе подстановка не сработает и в тексте буквально появится `${student.name}`.

Построчный разбор

- `document.getElementById("student-template")` — получаем сам `template` по его `id`.
- `document.getElementById("student-list")` — получаем элемент списка, куда будем добавлять карточки.
- `students.forEach((student) => {...})` — перебираем массив `students` методом `forEach`: он не изменяет массив, и число повторений равно длине массива.
- `studentTemplate.content.cloneNode(true)` — свойство `content` даёт доступ к содержимому шаблона, а метод `cloneNode(true)` клонирует его вместе со всеми вложенными элементами (в нашем случае — с несколькими `<p>` внутри `<li>`).
- `.querySelector(".student-name").textContent = ...` — находим внутри клона нужный элемент и записываем в него значение соответствующего поля студента. То же самое повторяется для всех остальных полей.
- `studentList.appendChild(studentClone)` — метод `appendChild` добавляет готовый `<li>` в конец списка `<ul>`.

Пример стилизации

```
#student-list {  
  display: flex;  
  gap: 30px;  
}  
  
.student-container {  
  display: inline-flex;  
  flex-direction: column;  
  gap: 7px;  
  border: 5px solid black;  
  padding: 10px;  
}
```